

Neural networks — Lab 2: optimizers, regularization, convolutions and attention

Lab 1 showed you a network that learns. This lab is about the choices that decide whether it learns *well*: how the optimizer walks through the loss landscape, how you stop a network from memorizing its training set, why convolutions suit spectra and images, and how attention lets a network decide for itself which parts of the input to compare.

Everything runs on synthetic spectra that we generate ourselves, so the lab needs no download and every experiment finishes in seconds on a CPU. All networks are written in PyTorch with the training loop visible; nothing is hidden in a `fit()` call.

Outline

1. Optimizers on a two-parameter loss landscape (numpy)
2. A small network fits a spectrum: SGD, momentum, Adam, activation functions and initialization
3. Overfitting on a small spectral data set and four cures: weight decay, dropout, early stopping, augmentation
4. Convolutions: hand-made kernels, a 1-D CNN, and why it is robust to calibration drift
5. Attention: the formula in numpy, attention as a soft library lookup, a tiny transformer
6. (Optional) An autoencoder rediscovers the rank of a transient-absorption matrix

Cells marked **Task** contain `# your code here`. The rest is given; read it, it is part of the lesson.

Part 0 — Setup

The cell below works on Colab and on a local machine. It fixes the random seeds so that your numbers are reproducible; different seeds give slightly different numbers, which is itself a lesson about the randomness of training.

```
In [ ]: import sys, time, math
import numpy as np
import matplotlib.pyplot as plt
import torch, torch.nn as nn, torch.nn.functional as F
from scipy import optimize, signal

torch.set_num_threads(2)
def set_seed(seed=0):
    np.random.seed(seed); torch.manual_seed(seed)
set_seed(0)
rng = np.random.default_rng(0)
print("numpy", np.__version__, "| torch", torch.__version__, "| Colab:",
```

Part 1 — Optimizers on a two-parameter landscape

A neural network has 10^4 to 10^{11} parameters, so we can never look at its loss landscape. We can look at a two-parameter one. The Rosenbrock function

$$L(x, y) = (1 - x)^2 + b(y - x^2)^2$$

has a single minimum at $(1, 1)$ that lies at the bottom of a long curved valley. Along the valley the loss barely changes; across it the loss rises steeply. This is exactly the situation in a network: some parameter directions are stiff, others are almost flat. Such a problem is called **ill-conditioned**.

Every optimizer we use in deep learning is a rule of the form

$$w_{t+1} = w_t - \eta \cdot (\text{something built from the gradients}).$$

Plain gradient descent uses the current gradient; the other rules remember previous gradients.

```
In [ ]: B = 100.0
def rosenbrock(w):
    x, y = w
    return (1 - x)**2 + B*(y - x**2)**2

def rosenbrock_grad(w):
    x, y = w
    return np.array([-2*(1 - x) - 4*B*x*(y - x**2),
                    2*B*(y - x**2)])

def run_optimizer(step, w0, lr, n_steps=3000, **kw):
    '''Apply `step(w, g, lr, state, t, **kw)` repeatedly. Returns the path
    w = np.array(w0, dtype=float); state = {}; path = [w.copy()]
    for t in range(1, n_steps + 1):
        g = rosenbrock_grad(w)
        w = step(w, g, lr, state, t, **kw)
        path.append(w.copy())
        if not np.all(np.isfinite(w)) or np.abs(w).max() > 1e3:
            break # diverged
    return np.array(path)

def gd_step(w, g, lr, state, t):
    return w - lr * g
```

```
def plot_paths(paths, title=''):
    xs, ys = np.linspace(-2, 2, 300), np.linspace(-1, 3, 300)
    XX, YY = np.meshgrid(xs, ys)
    ZZ = rosenbrock((XX, YY))
    fig, ax = plt.subplots(figsize=(7, 5))
    ax.contour(XX, YY, np.log10(ZZ + 1e-3), levels=25, cmap='Greys', line
    for name, p in paths.items():
        ax.plot(p[:, 0], p[:, 1], '.-', ms=2, lw=0.8, label=f'{name} ({le
    ax.plot(1, 1, 'r*', ms=12, label='minimum')
    ax.set(xlabel='x', ylabel='y', title=title); ax.legend(fontsize=8)
    plt.tight_layout()

w0 = (-1.5, 1.5)
paths = {'GD, lr=2e-3': run_optimizer(gd_step, w0, lr=2e-3)}
plot_paths(paths, 'Plain gradient descent: fast down the wall, then a cra
```

Task 1.1 — momentum. Momentum keeps a running velocity v and moves along it:

$$v_t = \beta v_{t-1} + g_t, \quad w_{t+1} = w_t - \eta v_t \quad (\beta \approx 0.9).$$

Components of the gradient that flip sign from step to step (across the valley) cancel in v ; components that keep their sign (along the valley) add up. Implement `momentum_step`. The dictionary `state` persists between calls; use it to store `v`.

```
In [ ]: def momentum_step(w, g, lr, state, t, beta=0.9):
    # your code here: read v from state (zeros if absent), update it, sto
    if 'v' not in state:
        state['v'] = np.zeros_like(w)
    # get v_{t-1}
    v_previous = state['v']
    # compute new velocity: v_t = beta * v_{t-1} + g_t
    v_t = beta * v_previous + g
    # save v_t back into state for the next step
    state['v'] = v_t
    # return updates wights
    return w - lr * v_t

paths['Momentum, lr=2e-3'] = run_optimizer(momentum_step, w0, lr=2e-3)
plot_paths(paths, 'Momentum: the same learning rate, ten times the progre
```

RMSProp (given) rescales every parameter by the running root-mean-square of its own gradient, so that stiff directions get small steps and flat directions large ones:

$$s_t = \beta_2 s_{t-1} + (1 - \beta_2) g_t^2, \quad w_{t+1} = w_t - \eta \frac{g_t}{\sqrt{s_t} + \epsilon}.$$

Task 1.2 — Adam. Adam combines both ideas and corrects for the fact that the running averages start at zero:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad s_t = \beta_2 s_{t-1} + (1 - \beta_2) g_t^2,$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{s}_t = \frac{s_t}{1 - \beta_2^t}, \quad w_{t+1} = w_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{s}_t} + \epsilon},$$

with the defaults $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$. Implement `adam_step`. The step counter `t` starts at 1.

```
In [ ]: def rmsprop_step(w, g, lr, state, t, beta2=0.9, eps=1e-8):
    s = state.get('s', np.zeros_like(w))
    s = beta2 * s + (1 - beta2) * g**2
    state['s'] = s
    return w - lr * g / (np.sqrt(s) + eps)

In [ ]: def adam_step(w, g, lr, state, t, beta1=0.9, beta2=0.999, eps=1e-8):
    # your code here
    m = state.get('m', np.zeros_like(w)) # first moment
    s = state.get('s', np.zeros_like(w)) # second moment

    # update
    m = beta1 * m + (1 - beta1)*g
    s = beta2 * s + (1 - beta2)*g**2
    # save back to step
    state['m'] = m
    state['s'] = s

    # compute bias-corrected estimates
    m_hat = m / (1 - beta1)
    s_hat = s / (1 - beta2)
    # return updates wights
    return w - lr * m_hat / (np.sqrt(s_hat) + eps)

paths['RMSProp, lr=1e-2'] = run_optimizer(rmsprop_step, w0, lr=1e-2)
paths['Adam, lr=5e-2'] = run_optimizer(adam_step, w0, lr=5e-2)
plot_paths(paths, 'Four optimizers on the same landscape')
```

Notice that RMSProp and Adam tolerate a 5 to 25 times larger learning rate than gradient descent; that is the practical reason Adam is the default. Notice also that near the minimum they do not settle as cleanly as momentum: their step is about η per parameter whatever the gradient, so they hover at a distance of order η from the minimum until the learning rate is lowered. That is one reason optimizers are combined with a learning-rate schedule (Part 2).

Task 1.3 — the learning rate is the most important hyperparameter. Run plain gradient descent from `w0` with learning rates 0.5e-3, 2e-3, 4e-3 and 6e-3 for 2000 steps and plot the loss against the iteration number on a logarithmic axis (`ax.semilogy`). One of them oscillates without converging and one diverges; `run_optimizer` stops when that happens.

```
In [ ]: fig, ax = plt.subplots(figsize=(6, 4))
for lr in [0.5e-3, 2e-3, 4e-3, 6e-3]: # different learning rates
    # your code here: run gd_step, compute the loss along the path, semilog
    # obtain path of (x, y) points along the optimization trajectory
    path = run_optimizer(gd_step, w0, lr=lr)
    # compute losses
    losses = rosenbrock(path.T)
    ax.semilogy(losses, label=f'lr={lr}')

ax.set(xlabel='iteration', ylabel='loss', title='Gradient descent: too slow')
plt.tight_layout()
```

Question: Describe in one sentence what you see.

Answer:

The plot shows how loss changes over iterations for four learning rates, where $lr = 0.002$ is optimal because smaller rates converge too slowly while larger ones oscillate or explodes.

Why curve fitting does not use gradient descent, and why deep learning cannot use curve fitting

When you fit a decay curve with `scipy.optimize.least_squares` or with KiMoPack, the fitter does not walk downhill with a fixed step. It uses the Jacobian $J_{ij} = \partial r_i / \partial p_j$ of the residuals to build the Gauss–Newton approximation of the Hessian, $H \approx J^\top J$, and solves a linear system for the step. Curvature information turns a valley into a direct route. The cell below shows this: `least_squares` reaches the minimum in a few dozen function evaluations, where gradient descent needed thousands of steps.

Why not do that for a network? For P parameters, $J^\top J$ is a $P \times P$ matrix. With $P = 10^6$ it has 10^{12} entries (4 TB in single precision), and solving the linear system would cost P^3 operations. Gradient descent needs only the gradient, which back-propagation delivers at the cost of about two forward passes, whatever P is. That trade — no curvature, but linear cost per step — is the whole reason for momentum, RMSProp and Adam: they try to recover a little curvature information from the history of gradients.

```
In [ ]: def residuals(w):
    x, y = w
    return np.array([1 - x, np.sqrt(B) * (y - x**2)]) # rosenbrock = su

res = optimize.least_squares(residuals, w0, method='lm')
print(f'least_squares (Levenberg-Marquardt): minimum at {res.x}, {res.nfe}')
print(f'gradient descent (lr=2e-3): final point {paths["GD", lr=2e-3]}
```

Part 2 — A small network fits a spectrum

Before we classify anything, we use a network as a flexible fitting function: one input (the wavelength), one output (the absorbance), two hidden layers. This is the same task as fitting a sum of Gaussians, except that the network does not know that the

peaks are Gaussians. It is the simplest setting in which to watch the optimizer, the activation function and the initialization at work. Here the training loop is written out in full:

1. `optimizer.zero_grad()` — clear the gradients from the previous step (PyTorch accumulates them),
2. forward pass and loss,
3. `loss.backward()` — back-propagation,
4. `optimizer.step()` — one update rule from Part 1.

```
In [ ]: x_grid = torch.linspace(0, 1, 200).unsqueeze(1) # (200, 1): 200
def spectrum(x):
    return (1.0*torch.exp(-0.5*((x-0.25)/0.04)**2) + 0.6*torch.exp(-0.5*(
        + 0.3*torch.exp(-0.5*((x-0.80)/0.03)**2))
set_seed(0)
y_grid = spectrum(x_grid) + 0.02*torch.randn_like(x_grid)

def make_mlp(activation=nn.Tanh, hidden=32, init_std=None):
    '''1 -> hidden -> hidden -> 1. If init_std is given, all weights are
    m = nn.Sequential(nn.Linear(1, hidden), activation(), nn.Linear(hidden, 1))
    if init_std is not None:
        for layer in m:
            if isinstance(layer, nn.Linear):
                nn.init.normal_(layer.weight, std=init_std); nn.init.zeros_(layer.bias)
    return m

def fit(model, optimizer, n_steps=3000, scheduler=None):
    history = []
    for step in range(n_steps):
        optimizer.zero_grad()
        loss = F.mse_loss(model(x_grid), y_grid)
        loss.backward()
        optimizer.step()
        if scheduler is not None:
            scheduler.step()
        history.append(loss.item())
    return history

def show_fit(models, title=''):
    fig, ax = plt.subplots(figsize=(7, 4))
    ax.plot(x_grid, y_grid, 'k.', ms=3, label='data')
    with torch.no_grad():
        for name, m in models.items():
            ax.plot(x_grid, m(x_grid), lw=1.5, label=name)
    ax.set(xlabel='x (wavelength, arbitrary)', ylabel='y (absorbance, arbitrary)')

set_seed(0); mlp_sgd = make_mlp()
hist_sgd = fit(mlp_sgd, torch.optim.SGD(mlp_sgd.parameters(), lr=0.1))
show_fit({'SGD, lr=0.1, 3000 steps': mlp_sgd}, 'Plain SGD after 3000 steps')
print('final MSE', hist_sgd[-1])
```

Task 2.1 — optimizers, again, but now on a real network. Train three fresh networks (`make_mlp()` , seed 0 each time) for 3000 steps with

- `torch.optim.SGD(lr=0.1)` ,
- `torch.optim.SGD(lr=0.1, momentum=0.9)` ,

- `torch.optim.Adam(lr=1e-2)`,

and plot the three loss histories on one logarithmic axis. Then show the three fits with `show_fit`. Which optimizer resolves the narrow peak at $x = 0.8$?

```
In [ ]: fits = {}
fig, ax = plt.subplots(figsize=(6, 4))
# your code here: for each optimizer, set_seed(0), build model, fit, ax.s
histories = {} # store loss histories
models = {} # stores models
# Model with optimizer 1: SGD
set_seed(0) # set seed to zero to ensures the results are reproducible
mlp_sgd = make_mlp()
opt_sgd = torch.optim.SGD(mlp_sgd.parameters(), lr=0.1)
hist_sgd = fit(mlp_sgd, opt_sgd, n_steps=3000)
ax.semilogy(hist_sgd, label='SGD, lr=0.1')
fits['SGD, lr=0.1'] = mlp_sgd

# Model with optimizer 2: SGD with Momentum
set_seed(0)
mlp_sgd_momentum = make_mlp()
opt_sgd_momentum = torch.optim.SGD(mlp_sgd_momentum.parameters(), lr=0.1,
hist_sgd_momentum = fit(mlp_sgd_momentum, opt_sgd_momentum, n_steps=3000)
ax.semilogy(hist_sgd_momentum, label='SGD+Momentum, lr=0.1')
fits['SGD+Momentum, lr=0.1'] = mlp_sgd_momentum

# Model with optimizer 3: Adam
mlp_adam = make_mlp()
opt_adam = torch.optim.Adam(mlp_adam.parameters(), lr=0.1)
hist_adam = fit(mlp_adam, opt_adam, n_steps=3000)
ax.semilogy(hist_adam, label='Adam, lr=1e-2')
fits['Adam, lr=1e-2'] = mlp_adam

ax.set(xlabel='Iteration', ylabel='Loss', title='Optimizer Loss History')
ax.legend(), plt.tight_layout(), plt.show()
show_fit(fits, 'Same network, same data, three optimizers')
```

Question: Which optimizer resolves the narrow peak at $x = 0.8$?

Answer:

None of the optimizers resolved the narrow peak at $x = 0.8$ within 3,000 steps. While plain SGD (blue) passes closest to the data points at $x = 0.8$, it is simply a smooth decay line that misses the shape entirely. SGD+Momentum and Adam correctly fit the first two peaks and the valley near $x = 0.7$, but none of the models developed the sharp, high-frequency bump required to resolve the third peak at $x = 0.8$.

Task 2.2 — the activation function decides the shape of the fit. Train two networks with 64 hidden units (`make_mlp(nn.Tanh, hidden=64)` and `make_mlp(nn.ReLU, hidden=64)`) with Adam (`lr=1e-2`, 4000 steps). Plot both fits and zoom into $x \in [0.7, 0.9]$. A ReLU network is a piecewise **linear** function of its input, with one kink per hidden unit at most; a tanh network is smooth. Which one would you trust for interpolation between measured points, and which one for a very large model?

```

In [ ]: # your code here
# Model 1
set_seed(0)
mlp_tahn = make_mlp(nn.Tanh, hidden=64)
opt_tahn = torch.optim.Adam(mlp_tahn.parameters(), lr=1e-2)
hist_tahn = fit(mlp_tahn, opt_tahn, n_steps=4000)

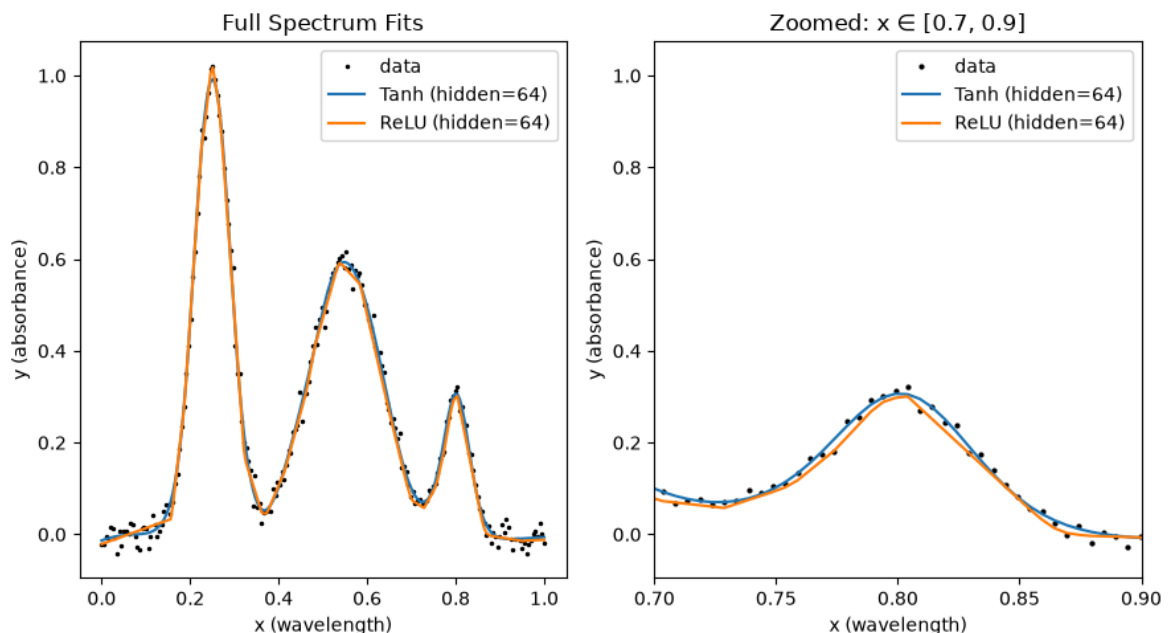
# Model 2
set_seed(0)
mlp_reLU = make_mlp(nn.ReLU, hidden=64)
opt_reLU = torch.optim.Adam(mlp_reLU.parameters(), lr=1e-2)
hist_reLU = fit(mlp_reLU, opt_reLU, n_steps=4000)

fits = {'Tanh (hidden=64)': mlp_tahn, 'ReLU (hidden=64)': mlp_reLU}
fig, ax = plt.subplots(1,2, figsize=(9,5))

ax[0].plot(x_grid, y_grid, 'k.', ms=3, label='data')
with torch.no_grad():
    for name, model in fits.items():
        ax[0].plot(x_grid, model(x_grid), lw=1.5, label=name)
ax[0].set(xlabel='x (wavelength)', ylabel='y (absorbance)', title='Full S
ax[1].plot(x_grid, y_grid, 'k.', ms=4, label='data')
with torch.no_grad():
    for name, model in fits.items():
        ax[1].plot(x_grid, model(x_grid), lw=1.5, label=name)
ax[1].set_xlim(0.7, 0.9)
ax[1].set(xlabel='x (wavelength)', ylabel='y (absorbance)', title='Zoomed
ax[1].legend()

plt.tight_layout()
plt.show()

```



Question: Which one would you trust for interpolation between measured points, and which one for a very large model?

Answer:

For interpolation between measured points, I would trust Tanh. The spectra shows Tanh as a smooth, continuous curves whereas ReLU produces linear segments with

sharp "kinks" between data points. But for a large model, I would trust ReLU. Large networks with many layers suffer from vanishing gradients when using Tanh, whereas ReLU maintains a constant gradient of 1 for positive inputs.

Task 2.3 — initialization. PyTorch initializes a linear layer with weights of order $1/\sqrt{n_{in}}$ (Kaiming uniform). Build a tanh network with `init_std=0.01` instead and train it with `SGD(lr=0.1, momentum=0.9)`. Compare its loss history with the default-initialized network from Task 2.1. Then run the helper `gradient_norms` on both networks *before* training to see why: with tiny weights each layer shrinks the signal, so the gradient of the first layer is tiny, and training stalls. This is the vanishing-gradient problem in its smallest form.

```
In [53]: def gradient_norms(model):
    '''Norm of the loss gradient for each weight matrix, at the current p
    model.zero_grad()
    F.mse_loss(model(x_grid), y_grid).backward()
    return {f'layer {i}': f'{layer.weight.grad.norm().item():.2e}' for i,
```

```
In [ ]: # your code here
set_seed(0)
mlp_default = make_mlp() # make model with normal default Kaiming initial
gn_default = gradient_norms(mlp_default) # check gradient norm

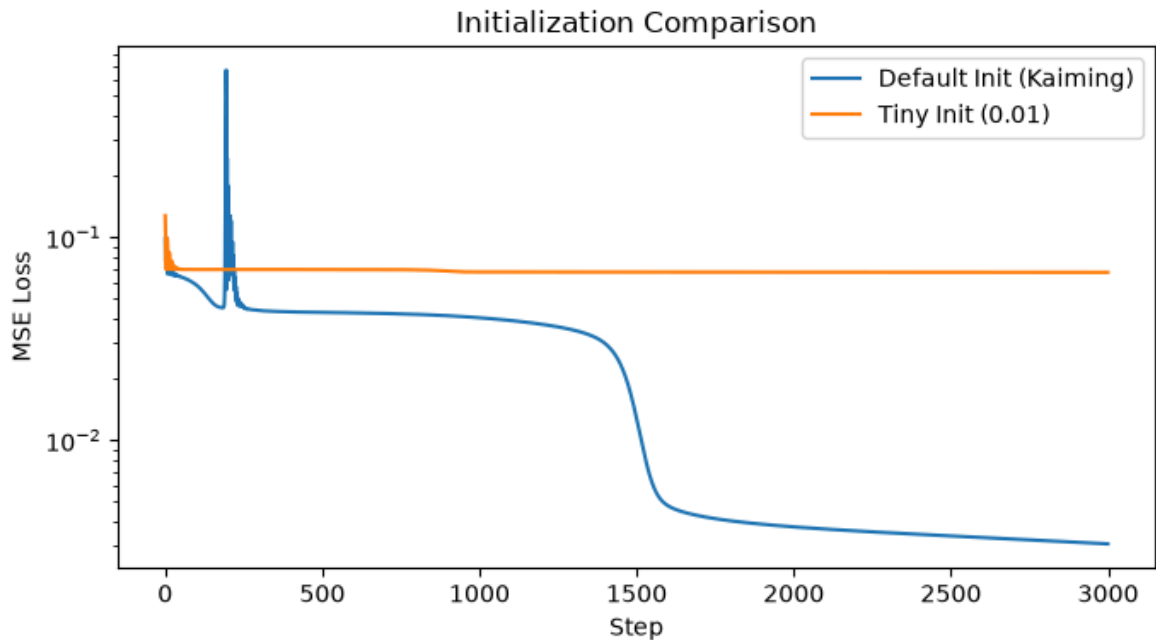
set_seed(0)
mlp_tiny_w = make_mlp(init_std=0.01) # make model with tiny weight initia
gn_tiny_w = gradient_norms(mlp_tiny_w)

print("Default Init (Kaiming):", gn_default)
print("Tiny Init (std=0.01): ", gn_tiny_w)

# train both networks with SGD + Momentum
set_seed(0)
opt_default = torch.optim.SGD(mlp_default.parameters(), lr=0.1, momentum=
hist_default = fit(mlp_default, opt_default, n_steps=3000)
set_seed(0)
opt_tiny_w = torch.optim.SGD(mlp_tiny_w.parameters(), lr=0.1, momentum=0.
hist_tiny_w = fit(mlp_tiny_w, opt_tiny_w, n_steps=3000)

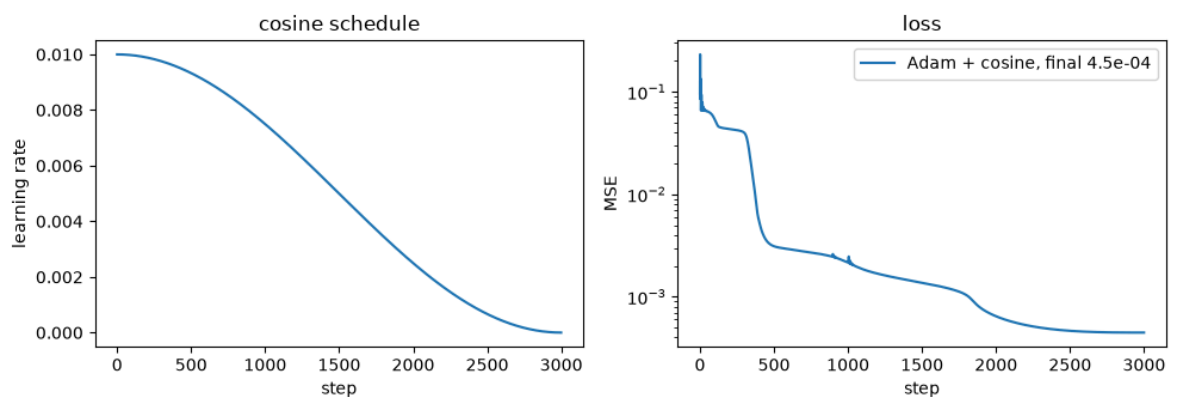
# plot result
fig, ax = plt.subplots(figsize=(7, 4))
ax.semilogy(hist_default, label='Default Init (Kaiming)')
ax.semilogy(hist_tiny_w, label='Tiny Init (0.01)')
ax.set(xlabel='Step', ylabel='MSE Loss', title='Initialization Comparison')
ax.legend()
plt.tight_layout()
plt.show()
```

```
Default Init (Kaiming): {'layer 0': '2.88e-02', 'layer 2': '5.43e-01', 'la
yer 4': '5.11e-01'}
Tiny Init (std=0.01): {'layer 0': '8.51e-04', 'layer 2': '7.00e-04', 'l
ayer 4': '6.56e-04'}
```



Learning-rate schedules (given). In Lab 1 the numpy network multiplied its learning rate by 0.999 after every iteration. That was a schedule: large steps early to cross the landscape, small steps late to settle into the minimum. PyTorch has them as `torch.optim.lr_scheduler`. The cosine schedule is the current default in many papers.

```
In [62]: set_seed(0); m_cos = make_mlp()
opt = torch.optim.Adam(m_cos.parameters(), lr=1e-2)
sched = torch.optim.lr_scheduler.CosineAnnealingLR(opt, T_max=3000)
lrs = []
h_cos = []
for step in range(3000):
    opt.zero_grad(); loss = F.mse_loss(m_cos(x_grid), y_grid); loss.backward()
    lrs.append(sched.get_last_lr()[0]); h_cos.append(loss.item())
fig, axs = plt.subplots(1, 2, figsize=(10, 3.5))
axs[0].plot(lrs); axs[0].set(xlabel='step', ylabel='learning rate', title='cosine schedule')
axs[1].semilogy(h_cos, label=f'Adam + cosine, final {h_cos[-1]:.1e}')
axs[1].set(xlabel='step', ylabel='MSE', title='loss'); axs[1].legend(); p
```



Part 3 — Overfitting on a small spectral data set, and four cures

We now turn to classification. The data are synthetic spectra of four "compounds", each with a characteristic multiplet pattern (a narrow doublet, a wide doublet, a broad singlet, a triplet). Every spectrum has a random amplitude, a small baseline offset and tilt, white noise, and a **calibration drift**: the whole pattern is shifted by up to `shift_max` points. Real spectrometers do that too.

We train on only **60 spectra** on purpose. A network with 10^5 parameters will memorize 60 examples with ease; the question is what it does on the 1000 spectra it has never seen.

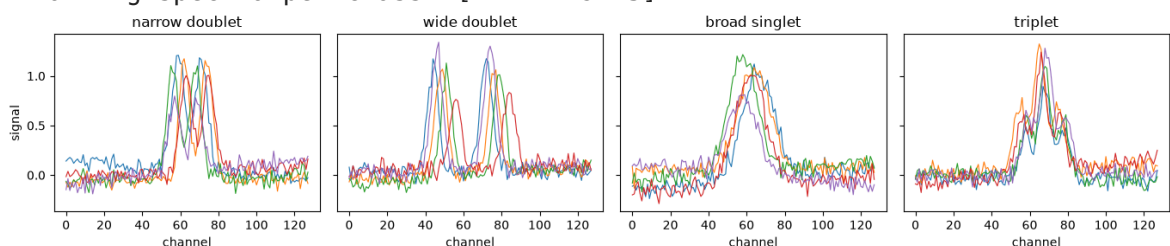
```
In [63]: TEMPLATES = {0: [(-6, 3, 1.0), (6, 3, 1.0)],           # narrow
          1: [(-14, 3, 1.0), (14, 3, 1.0)],                 # wide
          2: [(0, 8, 0.9)],                                   # broad
          3: [(-10, 3, 0.5), (0, 3, 1.0), (10, 3, 0.5)]     # triplet
        }
CLASS_NAMES = ['narrow doublet', 'wide doublet', 'broad singlet', 'triplet']
N_POINTS = 128

def make_spectra(n, rng, shift_max=6, noise=0.05, centre=64):
    x = np.arange(N_POINTS)
    X = np.zeros((n, N_POINTS), np.float32); y = rng.integers(0, 4, n)
    for i in range(n):
        c0 = centre + rng.integers(-shift_max, shift_max + 1)  #
        amp = rng.uniform(0.7, 1.3)
        for (dc, w, h) in TEMPLATES[y[i]]:
            X[i] += amp * h * np.exp(-0.5 * ((x - (c0 + dc)) / (w * rng.u
            X[i] += rng.uniform(-0.1, 0.1) + rng.uniform(-0.002, 0.002) * (x
            X[i] += noise * rng.standard_normal(N_POINTS)
    return X, y

rng = np.random.default_rng(0)
X_train, y_train = make_spectra(60, rng)
X_val, y_val = make_spectra(400, rng)
X_test, y_test = make_spectra(1000, rng)
X_shift, y_shift = make_spectra(1000, rng, shift_max=16)  # larger dr
T = lambda a: torch.as_tensor(a)
X_train_t, y_train_t, X_val_t, y_val_t, X_test_t, y_test_t, X_shift_t, y_

fig, axs = plt.subplots(1, 4, figsize=(14, 3), sharey=True)
for c, ax in enumerate(axs):
    for i in np.where(y_train == c)[0][:5]:
        ax.plot(X_train[i], lw=0.8)
    ax.set(title=CLASS_NAMES[c], xlabel='channel')
axs[0].set_ylabel('signal'); plt.tight_layout()
print('training spectra per class:', np.bincount(y_train))
```

training spectra per class: [14 12 19 15]



The model is a two-hidden-layer MLP with 256 units per layer (about 10^5 parameters). The training function keeps track of the training and validation loss after

every epoch. Note two details that Lab 1 did not have: `model.train()` / `model.eval()` switch dropout on and off, and `torch.no_grad()` turns off gradient tracking during evaluation.

Task 3.2 (prepare now, use later) — early stopping. The function has a `patience` argument. When it is set, training should stop once the validation loss has not improved for `patience` epochs, and the model should be restored to the parameters that had the best validation loss. Fill in the marked lines. Until you do, `patience` has no effect; the code still runs.

```
In [65]: class MLP(nn.Module):
    def __init__(self, p_dropout=0.0, hidden=256):
        super().__init__()
        self.net = nn.Sequential(nn.Linear(N_POINTS, hidden), nn.ReLU(),
                                nn.Linear(hidden, hidden), nn.ReLU(), nn
                                nn.Linear(hidden, 4))

    def forward(self, x):
        return self.net(x)

def n_params(model):
    return sum(p.numel() for p in model.parameters())

def accuracy(model, X, y):
    model.eval()
    with torch.no_grad():
        return (model(X).argmax(dim=1) == y).float().mean().item()

def train(model, optimizer, epochs=150, batch_size=32, augment=None, pati
    history = {'train': [], 'val': []}
    best_val, best_state, wait = np.inf, None, 0
    n = len(X_train_t)
    for epoch in range(epochs):
        model.train()
        perm = torch.randperm(n)
        for i in range(0, n, batch_size):
            idx = perm[i:i + batch_size]
            xb, yb = X_train_t[idx], y_train_t[idx]
            if augment is not None:
                xb = augment(xb)
            optimizer.zero_grad()
            loss = F.cross_entropy(model(xb), yb)
            loss.backward()
            optimizer.step()
        model.eval()
        with torch.no_grad():
            history['train'].append(F.cross_entropy(model(X_train_t), y_t
            val_loss = F.cross_entropy(model(X_val_t), y_val_t).item()
            history['val'].append(val_loss)
        if patience is not None:
            # your code here (Task 3.2): if val_loss improved, remember i
            # (clone each tensor) into best_state, reset wait; otherwise
            if val_loss < best_val: # # check if current validation loss
                best_val = val_loss
                wait = 0
                # Save a frozen snapshot of the weights so future trainin
                best_state = {k: v.clone() for k, v in model.state_dict()
            else:
```

```

        wait += 1 # increase wait or break
        if wait >= patience:
            print(f"Early stopping triggered at epoch {epoch}")
            break

    if best_state is not None:
        model.load_state_dict(best_state)
    return history

def plot_history(histories, title=''):
    fig, ax = plt.subplots(figsize=(7, 4))
    for i, (name, h) in enumerate(histories.items()):
        ax.plot(h['train'], '--', color=f'C{i}', lw=1, label=f'{name}: tr')
        ax.plot(h['val'], '-', color=f'C{i}', lw=1.5, label=f'{name}: val')
    ax.set(xlabel='epoch', ylabel='cross-entropy', title=title, ylim=(0,

print('MLP parameters:', n_params(MLP()))

```

MLP parameters: 99844

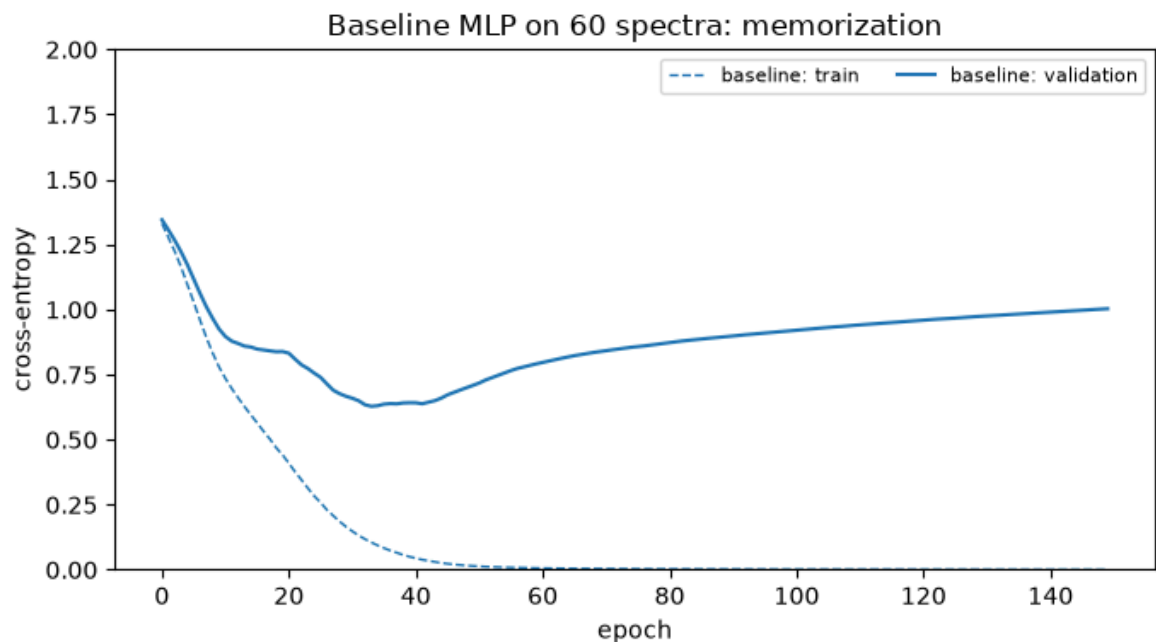
Baseline (given). Watch the two curves. The training loss goes to zero — the network has memorized 60 spectra. The validation loss first falls and then rises again: that rise is overfitting, and its onset marks where early stopping would stop. The expected loss of a network that knows nothing about four balanced classes is $\ln 4 \approx 1.39$; check that the curves start near that value.

```

In [84]: set_seed(0); baseline = MLP()
histories = {'baseline': train(baseline, torch.optim.Adam(baseline.parameters()),
results = {'baseline': (accuracy(baseline, X_test_t, y_test_t), accuracy(baseline, X_train_t, y_train_t))}
plot_history(histories, 'Baseline MLP on 60 spectra: memorization')
print(f'test accuracy {results["baseline"][0]:.3f} (start of val curve:

```

test accuracy 0.784 (start of val curve: 1.35, $\ln 4 = 1.39$)



Task 3.1 — weight decay and dropout. Train two more networks (seed 0, Adam, $\text{lr}=1\text{e-}3$, 150 epochs):

- `MLP()` with `torch.optim.Adam(..., weight_decay=1e-2)`. Weight decay shrinks every weight a little at each step; for plain SGD it is identical to an

L_2 penalty $\lambda \sum w^2$ on the loss.

- `MLP(p_dropout=0.5)` . Dropout zeroes half of the hidden units at random during each training step, so no unit can rely on a specific partner.

Add both to `histories` and `results` (test accuracy and shifted-test accuracy, as for the baseline) and plot all histories together. What happens to the rising part of the validation curve?

In [68]: *# your code here*

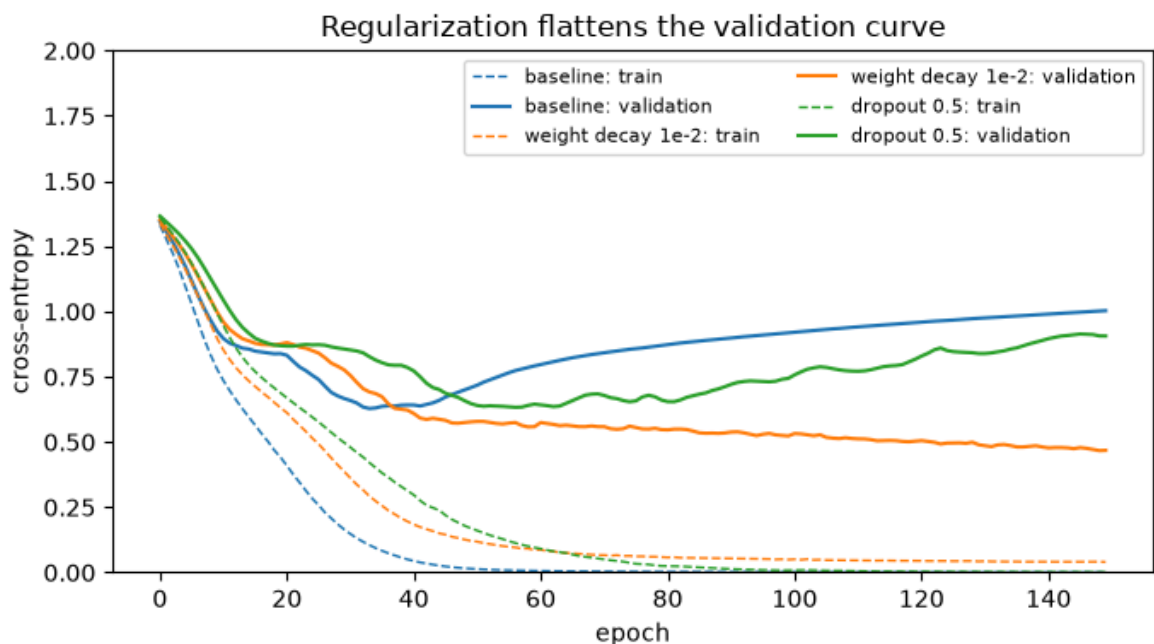
```
# weight decay Model
set_seed(0)
mlp_wd = MLP()
opt_wd = torch.optim.Adam(mlp_wd.parameters(), lr=1e-3, weight_decay=1e-2)
histories['weight decay 1e-2'] = train(mlp_wd, opt_wd)
results['weight decay 1e-2'] = (accuracy(mlp_wd, X_test_t, y_test_t), acc

# drop out Model
set_seed(0)
mlp_do = MLP(p_dropout=0.5)
opt_do = torch.optim.Adam(mlp_do.parameters(), lr=1e-3)
histories['dropout 0.5'] = train(mlp_do, opt_do)
results['dropout 0.5'] = (accuracy(mlp_do, X_test_t, y_test_t), accuracy(

# Plot all histories together
plot_history(histories, 'Regularization flattens the validation curve')

# Plot table
for name, (acc_test, acc_shift) in results.items():
    print(f'{name:20s} | Test Acc: {acc_test:.3f} | Shifted Test Acc: {acc
```

baseline	Test Acc: 0.784 Shifted Test Acc: 0.401
weight decay 1e-2	Test Acc: 0.866 Shifted Test Acc: 0.447
dropout 0.5	Test Acc: 0.806 Shifted Test Acc: 0.411



Task 3.2 — early stopping. If you have not done so, fill in the `patience` block in `train` above and re-run that cell. Then train a baseline `MLP()` with

`patience=15` and add it to `histories` and `results`. How many epochs did it run? Compare its test accuracy with the baseline.

```
In [ ]: # your code here
histories = {}
results = {}
set_seed(0)
mlp_baseline = MLP()
opt_baseline = torch.optim.Adam(mlp_baseline.parameters(), lr=1e-3)
histories['Baseline + Patience=15'] = train(mlp_baseline, opt_baseline, p
results['Baseline + Patience=15'] = (accuracy(mlp_baseline, X_test_t, y_t

# plot comparison
plot_history(histories, 'Task 3.2: Early Stopping Comparison')

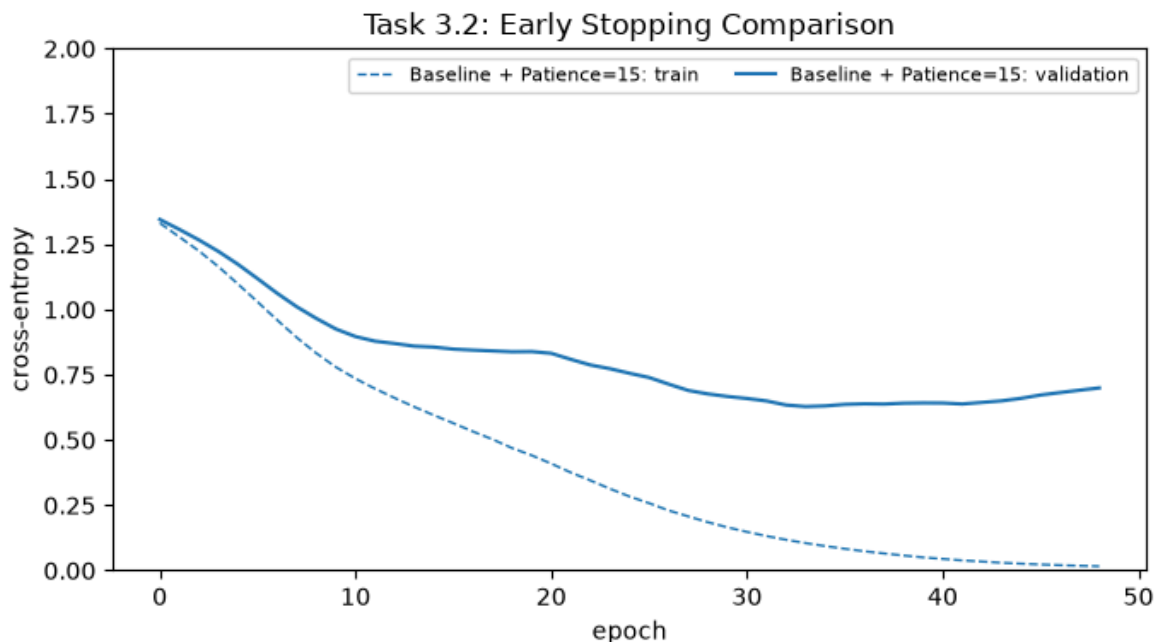
# print epoch count and accuracy comparison
epochs_run = len(histories['Baseline + Patience=15']['train'])
print(f"Early stopping triggered and stopped training at epoch {epochs_ru

print(f"{'Model':25s} | {'Test Acc':<10s} | {'Shifted Test Acc':<15s}")
print("-" * 55)
for name, (acc_test, acc_shift) in results.items():
    print(f"{'name':25s} | {'acc_test':<10.3f} | {'acc_shift':<15.3f}")
```

Early stopping triggered at epoch 48

Early stopping triggered and stopped training at epoch 49 / 150.

Model	Test Acc	Shifted Test Acc
Baseline + Patience=15	0.753	0.388



Question: How many epochs did it run? Compare its test accuracy with the baseline.

Answer:

It ran 49 epochs (0 through 48), stopping at less than one-third of the maximum 150 epochs.

The early-stopping model achieved a 75.3% test accuracy. Without early stopping, as training continues toward 150 epochs, the baseline network memorizes noise in the

training set and degrades test performance. Because patience=15 tracked the validation loss, the final model restored best_state from epoch 34 (where validation loss reached its minimum before 15 epochs of no improvement), successfully preventing overfitting.

Task 3.3 — augmentation: put the physics into the data. We know that the spectrometer drifts and that the noise is white. So we can manufacture new training spectra from old ones: shift each mini-batch by a random number of channels (`torch.roll(xb, shifts, dims=1)`) between -4 and +4 and add Gaussian noise with standard deviation 0.03. Write `augment(xb)`, train a baseline `MLP()` with `augment=augment` for 150 epochs and add it to `histories` and `results`.

Then print a table of all results with two columns: accuracy on the ordinary test set, and accuracy on the shifted test set (`shift_max=16`, larger than anything seen in training). Which cure helps on both? Why is that one different from the other three?

```
In [74]: def augment(xb):
# your code here
# random shift between -4 and +4 channels for each batch
shifts = torch.randint(-4, 5, (1,)).item()
xb_augmented = torch.roll(xb, shifts=shifts, dims=1)

# add Gaussian noise with std = 0.03
noise = torch.randn_like(xb_augmented) * 0.03
return xb_augmented + noise
set_seed(0)
mlp_aug = MLP()
opt_aug = torch.optim.Adam(mlp_aug.parameters(), lr=1e-3)

histories['augmentation'] = train(mlp_aug, opt_aug, epochs=150, augment=augment)
results['augmentation'] = (accuracy(mlp_aug, X_test_t, y_test_t), accuracy(
# your code here: train, store in histories['augmentation'] and results['

print(f'{"model":22s} {"test acc":>9s} {"shifted test acc":>17s}')
for k, (a, b) in results.items():
    print(f'{k:22s} {a:9.3f} {b:17.3f}')
```

model	test acc	shifted test acc
Baseline + Patience=15	0.753	0.388
augmentation	0.978	0.599

Question: Which cure helps on both? Why is that one different from the other three (Weight Decay, Dropout, Early Stopping)?

Answer:

Data Augmentation is the cure that helps on both test sets: (0.978, 0.599) compared to (0.753, 0.388).

It is different because the other three (Weight Decay, Dropout, and Early Stopping) are pure mathematical tricks—they only stop the model from over-memorizing the training data, but they still assume the test data will look identical to the training data.

Data Augmentation, on the other hand, actively teaches the model the physics of the problem. By shifting the data and adding noise during training, the network learns to

ignore small physical drifts, allowing it to perform well even when tested on large, unseen spectral shifts.

Part 4 — Convolutions

Augmentation taught the MLP that shifts do not matter, at the price of learning the same peak pattern separately at every position. A convolutional layer gets this for free: it slides a small **kernel** over the input and computes the same weighted sum at every position. The same pattern is detected wherever it appears. In deep-learning language the layer computes a cross-correlation,

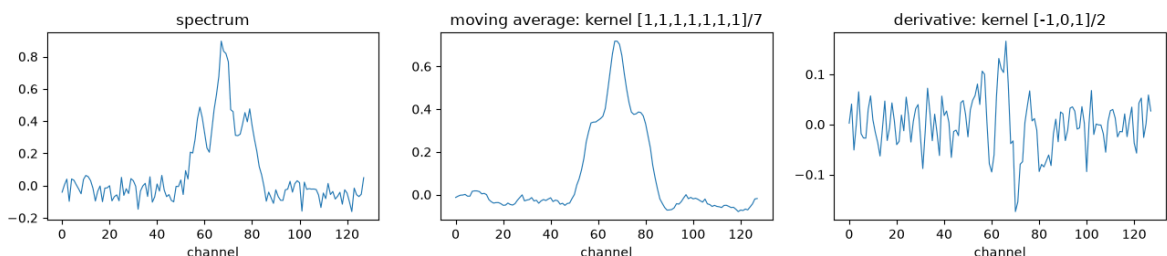
$$(x \star k)[i] = \sum_j x[i + j] k[j],$$

and the weights k are learned. A layer with C_{in} input channels, C_{out} output channels and kernel length K has $C_{\text{out}}(C_{\text{in}}K + 1)$ parameters, independent of the length of the spectrum. The dense first layer of our MLP had $128 \times 256 + 256 = 33,024$.

4.1 Kernels you already know

The first cell applies two hand-made kernels to a training spectrum: a moving average, and a central-difference derivative. You have used both before as smoothing and as the first derivative of a spectrum; Savitzky–Golay filters are exactly such kernels.

```
In [75]: s = X_train[np.where(y_train == 3)[0][0]] # one
smooth = np.ones(7) / 7
deriv = np.array([-1, 0, 1]) / 2
fig, axs = plt.subplots(1, 3, figsize=(13, 3))
axs[0].plot(s, lw=0.8); axs[0].set_title('spectrum')
axs[1].plot(np.correlate(s, smooth, mode='same'), lw=0.8); axs[1].set_title('moving average: kernel [1,1,1,1,1,1,1]/7')
axs[2].plot(np.correlate(s, deriv, mode='same'), lw=0.8); axs[2].set_title('derivative: kernel [-1,0,1]/2')
for ax in axs: ax.set_xlabel('channel')
plt.tight_layout()
```



Task 4.1 — write the convolution yourself. Implement `correlate1d(x, k)` with an explicit loop over output positions, for `mode='valid'` (output length `len(x) - len(k) + 1`, no padding). Check against `np.correlate(x, k, mode='valid')`. Then answer: what is the output length with padding of `(len(k)-1)//2` zeros on both sides, and what does a stride of 2 do to it?

```
In [77]: def correlate1d(x, k):
# your code here
```

```

'''1D cross-correlation with mode='valid'.'''
N, K = len(x), len(k)
out_len = N - K + 1
out = np.zeros(out_len)

for i in range(out_len):
    out[i] = np.sum(x[i : i + K] * k)

return out

# --- Verification against NumPy ---
np.random.seed(0)
x_test = np.random.randn(20)
k_test = np.array([1, 0, -1]) # Central difference derivative kernel

my_res = correlate1d(x_test, k_test)
np_res = np.correlate(x_test, k_test, mode='valid')

assert np.allclose(correlate1d(s, deriv), np.correlate(s, deriv, mode='va
print('ok')

```

ok

Question: what is the output length with padding of $(\text{len}(k)-1)//2$ zeros on both sides, and what does a stride of 2 do to it?

Answer:

The output length becomes N (exactly equal to the input length $\text{len}(x)$).

A stride of 2 halves the output length (downsampling/subsampling the signal by a factor of 2).

4.2 A two-dimensional kernel on an image (given)

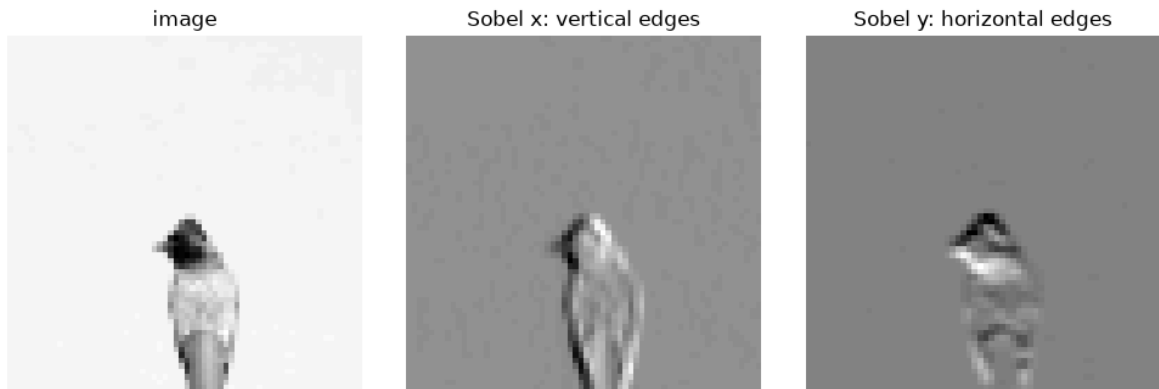
The same idea in two dimensions is the basis of every image network from Lab 1. A Sobel kernel responds to vertical edges; its transpose to horizontal ones. The first layer of a trained image CNN typically learns kernels that look like these, plus colour blobs.

```

In [78]: from pathlib import Path
img = None
for cand in [Path('train/bird'), Path('Bern02/Labs/Neural_Networks/train/
files = sorted(cand.glob('*.jpg')) if cand.exists() else []
if files:
    try:
        from PIL import Image
        img = np.asarray(Image.open(files[0]).convert('L'), dtype=flo
        break
    except Exception:
        continue # unre
if img is None: # no d
    yy, xx = np.mgrid[0:64, 0:64]
    img = ((xx - 40)**2 + (yy - 24)**2 < 150).astype(float) + 0.5 * ((xx
    sobel_x = np.array([[ -1, 0, 1], [-2, 0, 2], [-1, 0, 1]], float)
    fig, axs = plt.subplots(1, 3, figsize=(10, 3.3))
    for ax, im, t in zip(axs, [img, signal.correlate2d(img, sobel_x, mode='sa
        ['image', 'Sobel x: vertical edges', 'Sobel y: horiz

```

```
ax.imshow(im, cmap='gray'); ax.set_title(t); ax.axis('off')
plt.tight_layout()
```



4.3 A one-dimensional CNN for the spectra

The network below has three convolutional layers, two max-pooling steps and a **global average pool**: after the last convolution the 16 feature channels are averaged over all positions, and only these 16 numbers reach the classifier. The classifier therefore cannot know *where* a pattern was, only *whether* it was there — which is what we want for a drifting spectrometer. The receptive field of the last layer (how many input channels one output value sees) is $7 + 6 \cdot 2 + 6 \cdot 4 = 43$ channels, wide enough to cover the wide doublet.

Task 4.2 — write the forward pass. The layers are defined; write `forward`.

PyTorch's `Conv1d` expects the shape `(batch, channels, length)`, so start with `x.unsqueeze(1)`. Use `F.relu` and `F.max_pool1d(., 2)` after the first two convolutions, `F.relu` after the third, average over the length with `.mean(dim=2)` and apply `self.fc`. The comments give the shape after each step.

```
In [ ]: class CNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv1d(1, 8, kernel_size=7, padding=3) # 1 -> 8 c
        self.conv2 = nn.Conv1d(8, 16, kernel_size=7, padding=3) # 8 -> 16
        self.conv3 = nn.Conv1d(16, 16, kernel_size=7, padding=3) # 16 ->
        self.fc = nn.Linear(16, 4)

    def forward(self, x): # x: (batch, 128)
        # your code here
        # (batch, 1, 128) -> conv1+relu+pool -> (batch, 8, 64) -> conv2
        x = x.unsqueeze(1) # add channel dimension: (batch, length) -> (b

        # Conv1 -> ReLU -> MaxPool
        # (batch, 1, 128) -> (batch, 8, 128) -> (batch, 8, 64)
        x = F.max_pool1d(F.relu(self.conv1(x)), 2)

        # Conv2 -> ReLU -> MaxPool
        # (batch, 8, 64) -> (batch, 16, 64) -> (batch, 16, 32)
        x = F.max_pool1d(F.relu(self.conv2(x)), 2)

        # Conv3 -> ReLU
```

```

    # (batch, 16, 32) -> (batch, 16, 32)
    x = F.relu(self.conv3(x))

    # Global Average Pooling over the length dimension (dim=2)
    # (batch, 16, 32) -> (batch, 16)
    x = x.mean(dim=2)

    # Linear Classifier
    # (batch, 16) -> (batch, 4)
    return self.fc(x)

print('CNN parameters:', n_params(CNN()), ' MLP parameters:', n_params(ML

```

CNN parameters: 2852 MLP parameters: 99844

Task 4.3 — parameter count by hand. Compute the number of parameters of CNN from the formula $C_{\text{out}}(C_{\text{in}}K + 1)$ per convolution plus $16 \cdot 4 + 4$ for the classifier, and check it against `n_params(CNN())`.

```

In [ ]: # your code here
# compute number of paramters
p_conv1 = 8 * (1 * 7 + 1)
p_conv2 = 16 * (8 * 7 + 1)
p_conv3 = 16 * (16 * 7 + 1)
p_fc = 16 * 4 + 4

total_hand = p_conv1 + p_conv2 + p_conv3 + p_fc

print(f"Hand calculation: {total_hand}")
print(f"n_params(CNN()): {n_params(CNN())}")
assert total_hand == n_params(CNN()), "Mismatch in parameter count!"

```

Hand calculation: 2852

n_params(CNN()): 2852

Training, robustness and the learned kernels are given below. The CNN trains on the same 60 spectra without augmentation.

```

In [87]: set_seed(0); cnn = CNN()
hist_cnn = train(cnn, torch.optim.Adam(cnn.parameters(), lr=3e-3), epochs
results['1-D CNN (no augmentation)'] = (accuracy(cnn, X_test_t, y_test_t)
plot_history({'baseline MLP': histories['baseline'], '1-D CNN': hist_cnn})

# robustness against calibration drift
shifts = [0, 4, 8, 12, 16, 24, 32]
acc_curves = {name: [] for name in ['MLP baseline', 'MLP + augmentation',
for sm in shifts:
    Xs, ys = make_spectra(500, np.random.default_rng(5), shift_max=sm)
    for name, m in zip(acc_curves, [baseline, mlp_aug, cnn]):
        acc_curves[name].append(accuracy(m, T(Xs), T(ys)))
fig, ax = plt.subplots(figsize=(6, 4))
for name, a in acc_curves.items():
    ax.plot(shifts, a, 'o-', label=name)
ax.axvline(6, color='gray', ls=':', label='drift in training data'); ax.s
ax.legend(); ax.set_title('Translation equivariance built in beats transl

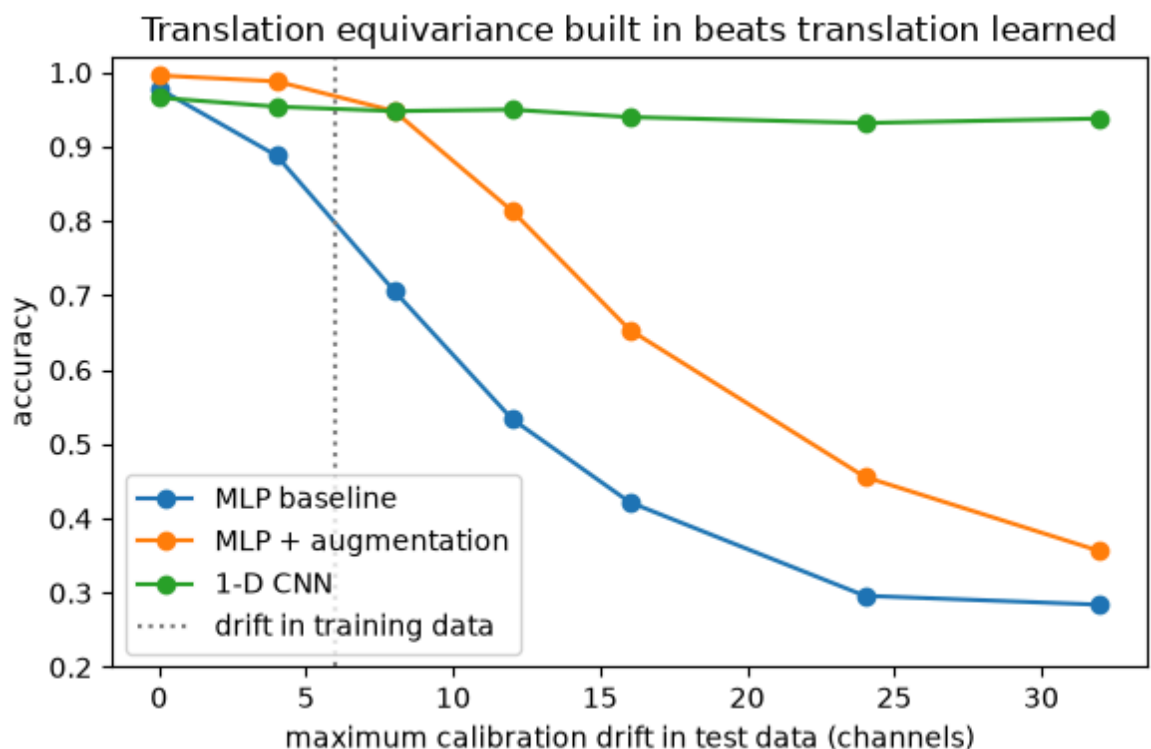
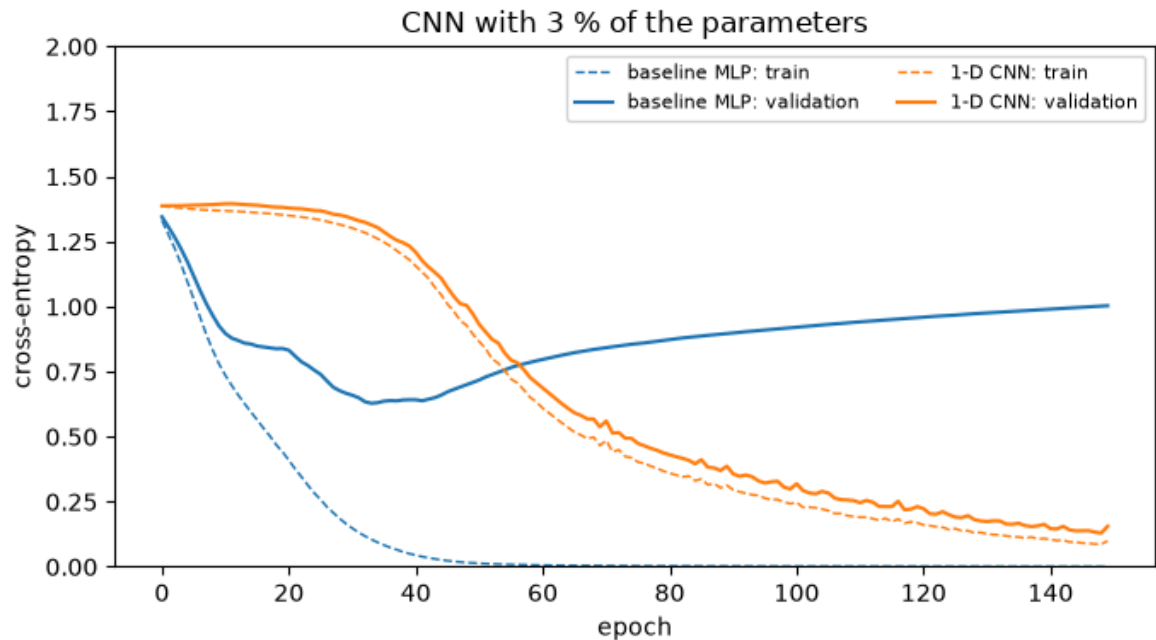
# learned first-layer kernels
fig, axs = plt.subplots(1, 8, figsize=(14, 1.8))
for i, ax in enumerate(axs):

```

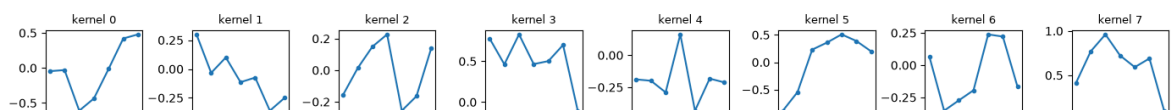
```
ax.plot(cnn.conv1.weight[i, 0].detach().numpy(), 'o-', ms=3); ax.set_
plt.suptitle('The 8 learned first-layer kernels: smoothers, edge detector

print(f'{"model":28s} {"test acc":>9s} {"shifted test acc":>17s}')
for k, (a, b) in results.items():
    print(f'{k:28s} {a:9.3f} {b:17.3f}')
```

model	test acc	shifted test acc
baseline	0.784	0.401
1-D CNN (no augmentation)	0.941	0.954



The 8 learned first-layer kernels: smoothers, edge detectors, and some noise



Part 5 — Attention

A convolution compares each position with its fixed neighbourhood. Attention lets every position decide, from the data, which other positions to look at. It is the operation inside every transformer, and therefore inside every large language model and inside AlphaFold.

5.1 The formula

Given n **queries** $Q \in \mathbb{R}^{n \times d}$, m **keys** $K \in \mathbb{R}^{m \times d}$ and m **values** $V \in \mathbb{R}^{m \times d_v}$,

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right) V.$$

Each row of $QK^\top$ contains the dot products of one query with all keys. The softmax turns them into weights that are positive and sum to one. The output for that query is the weighted average of the values. Dividing by $\sqrt{d}$ keeps the dot products of order one, so the softmax does not saturate.

Task 5.1 — implement `softmax` (along the last axis, subtracting the row maximum first for numerical stability) and `attention(Q, K, V)`, returning both the output and the weight matrix A . Check that every row of A sums to one.

```
In [89]: def softmax(z, axis=-1):
# your code here
'''Softmax along the last axis with numerical stability trick.'''
# Subtract max per row (keepdims=True keeps shape aligned for broadcast)
z_max = np.max(z, keepdims=True)
exp_z = np.exp(z - z_max)
return exp_z / np.sum(exp_z, axis=-1, keepdims=True)

def attention(Q, K, V):
# your code here: return output, A
'''
Scaled Dot-Product Attention.
Q: (n, d)
K: (m, d)
V: (m, d_v)
Returns: (output, A) where A is the (n, m) attention weight matrix
'''
d = Q.shape[-1]

# compute dot products scaled by sqrt(d)
# Q @ K.T gives shape (n, m)
scores = (Q @ K.T) / np.sqrt(d)

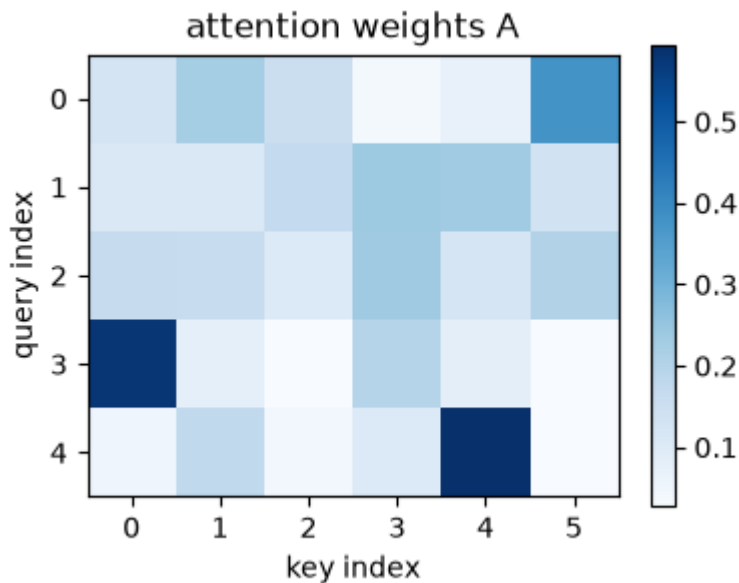
# compute attention weights via softmax along the last axis
A = softmax(scores)

# weighted average of values
# A @ V gives shape (n, d_v)
out = A @ V

return out, A

rng = np.random.default_rng(1)
```

```
Q, K, V = rng.standard_normal((5, 8)), rng.standard_normal((6, 8)), rng.s
out, A = attention(Q, K, V)
assert out.shape == (5, 3) and np.allclose(A.sum(axis=1), 1)
fig, ax = plt.subplots(figsize=(4, 3)); im = ax.imshow(A, cmap='Blues');
ax.set(xlabel='key index', ylabel='query index', title='attention weights
```



5.2 Attention as a soft library lookup (given)

Here is attention with a chemical meaning. The **keys** are the four noise-free template spectra of our compound classes (a spectral library). The **values** are their class labels as one-hot vectors. The **query** is a measured spectrum. QK^T is then the overlap of the measurement with each library entry, the softmax turns the overlaps into weights, and the output is a probability vector over the classes — a differentiable version of "look up the closest reference spectrum". The temperature (here the scale of the dot products) decides how sharp the lookup is.

```
In [90]: def template(c, centre=64):
    x = np.arange(N_POINTS); t = np.zeros(N_POINTS)
    for dc, w, h in TEMPLATES[c]:
        t += h * np.exp(-0.5 * ((x - (centre + dc)) / w)**2)
    return t
library = np.stack([template(c) for c in range(4)]) # keys:
library = library / np.linalg.norm(library, axis=1, keepdims=True) # values
labels_onehot = np.eye(4) # values

queries = X_test[:6] / np.linalg.norm(X_test[:6], axis=1, keepdims=True) # input
scale = 20.0 # inverse temperature
A_lib = softmax(scale * queries @ library.T)
probs = A_lib @ labels_onehot
print('true class      attention weights on the 4 library spectra      predicted')
for i in range(6):
    print(f'{CLASS_NAMES[y_test[i]]:15s}', np.array2string(A_lib[i], prec=3), y_test[i])
```

true class	attention weights on the 4 library spectra	predicted
wide doublet	[0.73 0.06 0.15 0.06]	narrow doublet
wide doublet	[0. 1. 0. 0.]	wide doublet
narrow doublet	[0.47 0. 0.43 0.1]	narrow doublet
broad singlet	[0.08 0. 0.59 0.32]	broad singlet
broad singlet	[0.09 0. 0.61 0.3]	broad singlet
narrow doublet	[0.96 0. 0.03 0.]	narrow doublet

Most rows are right, but not all: a wide doublet that happens to be shifted by several channels overlaps a narrow doublet template better than its own. A library lookup by dot product has no notion of translation; the CNN of Part 4 has. Keep that in mind when we train a transformer below: it has to learn shift tolerance from data, which is where its appetite for data comes from.

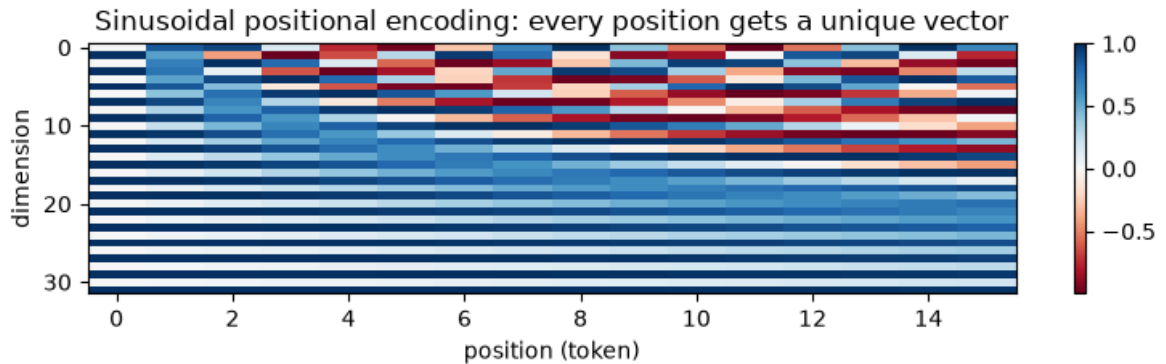
5.3 Attention does not know where things are (given)

Self-attention treats its input as a set: if you permute the tokens, the outputs are permuted in the same way and a mean over tokens is unchanged. For a spectrum this is a problem, because the position of a peak carries information. Transformers therefore add a **positional encoding** to every token before attention: a vector that differs from position to position. The original transformer used sines and cosines of increasing wavelength; many modern models learn the vector instead. The cell shows both facts.

```
In [91]: rng = np.random.default_rng(2)
X_tok = rng.standard_normal((6, 8)) # 6 tokens
Wq, Wk, Wv = [rng.standard_normal((8, 8)) / np.sqrt(8) for _ in range(3)]
def self_attention(Xt): return attention(Xt @ Wq, Xt @ Wk, Xt @ Wv)[0]
perm = rng.permutation(6)
out_a = self_attention(X_tok); out_b = self_attention(X_tok[perm])
print('permuting the tokens permutes the output in the same way:', np.all(out_a == out_b))
print('the mean over tokens is therefore identical:', np.allclose(out_a.mean(0), out_b.mean(0)))

def sinusoidal_pe(n_positions, d, base=10000.0):
    '''Vaswani et al. 2017. base sets the longest wavelength; 10000 suits
    for our 16 patches a base of 100 uses all dimensions.'''
    pos = np.arange(n_positions)[:, None]; i = np.arange(d // 2)[:, None]
    angle = pos / base ** (2 * i / d)
    pe = np.zeros((n_positions, d)); pe[:, 0::2] = np.sin(angle); pe[:, 1::2] = np.cos(angle)
    return pe
fig, ax = plt.subplots(figsize=(8, 2.5))
im = ax.imshow(sinusoidal_pe(16, 32, base=100).T, aspect='auto', cmap='RdYlBu')
ax.set(xlabel='position (token)', ylabel='dimension', title='Sinusoidal positional encoding')
```

```
permuting the tokens permutes the output in the same way: True
the mean over tokens is therefore identical: True
```



5.4 A tiny transformer for the spectra

We cut every spectrum into 16 patches of 8 channels, embed each patch linearly into a 32-dimensional token, add a learned positional encoding, and apply one transformer block: self-attention with a residual connection, then a small MLP with a residual connection, each preceded by layer normalization. The tokens are averaged and classified. This is a one-block, one-head version of the vision transformer, and the same design (without the patching) as in a language model.

Task 5.2 — write the self-attention module. In `forward`, compute Q, K, V with the three linear layers, the weights $A = \text{softmax}(Q K^T / \sqrt{d})$ along the last dimension (use `torch.softmax` and `K.transpose(1, 2)` for the batched transpose), and return $A @ V$ together with A .

```
In [ ]: class SelfAttention(nn.Module):
    def __init__(self, d):
        super().__init__()
        self.q, self.k, self.v = nn.Linear(d, d), nn.Linear(d, d), nn.Linear(d, d)
        self.d = d
    def forward(self, x):
        # your code here
        # linear projections
        Q = self.q(x) # (batch, 16, 32)
        K = self.k(x) # (batch, 16, 32)
        V = self.v(x) # (batch, 16, 32)
        # scaled dot-product scores: (batch, 16, 32) @ (batch, 32, 16) ->
        scores = torch.matmul(Q, K.transpose(1, 2)) / math.sqrt(self.d)
        # softmax along the last dimension (over key positions)
        A = torch.softmax(scores, dim=-1)
        # weighted combination of values: (batch, 16, 16) @ (batch, 16, 32)
        out = A @ V
        return out, A

class TinyTransformer(nn.Module):
    def __init__(self, patch=8, d=32, n_classes=4):
        super().__init__()
        self.patch = patch; n_tokens = N_POINTS // patch
        self.embed = nn.Linear(patch, d)
        self.pos = nn.Parameter(0.02 * torch.randn(1, n_tokens, d))
        self.attn = SelfAttention(d)
        self.norm1, self.norm2 = nn.LayerNorm(d), nn.LayerNorm(d)
        self.mlp = nn.Sequential(nn.Linear(d, 2 * d), nn.ReLU(), nn.Linear(2 * d, d))
        self.head = nn.Linear(d, n_classes)
```

```

def forward(self, x, return_attention=False):
    tokens = x.view(x.shape[0], -1, self.patch)           # (batch, 16
    h = self.embed(tokens) + self.pos                     # (batch, 16
    a, A = self.attn(self.norm1(h)); h = h + a             # attention
    h = h + self.mlp(self.norm2(h))                       # MLP + resi
    out = self.head(h.mean(dim=1))
    return (out, A) if return_attention else out

set_seed(0); tt = TinyTransformer()
_, A_test = tt(X_test_t[:2], return_attention=True)
assert A_test.shape == (2, 16, 16) and torch.allclose(A_test.sum(-1), tor
print('Transformer parameters:', n_params(tt))

```

Transformer parameters: 8420

Data hunger (given). We train the transformer twice: on the 60 spectra from Part 3, and on 2000 spectra. Attention has almost no built-in assumptions about the data — it must learn from examples that neighbouring patches belong together and that a shifted pattern is the same pattern. The CNN has both assumptions built in. With 60 spectra the CNN wins; with 2000 the transformer catches up on the test set but still fails on drifts it has never seen, because a learned positional encoding is not translation equivariant. That is the trade-off in one table: **inductive bias buys data efficiency; flexibility buys generality once data are plentiful.**

```

In [95]: set_seed(0); tt_small = TinyTransformer()
hist_tt = train(tt_small, torch.optim.Adam(tt_small.parameters(), lr=1e-3
results['transformer, 60 spectra'] = (accuracy(tt_small, X_test_t, y_test

# 2000 training spectra: swap the training tensors, train, swap back
X_big, y_big = make_spectra(2000, np.random.default_rng(7))
X_train_t, y_train_t, X_small_t, y_small_t = T(X_big), T(y_big), X_train_
set_seed(0); tt_big = TinyTransformer()
t0 = time.time(); hist_tt_big = train(tt_big, torch.optim.Adam(tt_big.par
results['transformer, 2000 spectra'] = (accuracy(tt_big, X_test_t, y_test
set_seed(0); cnn_big = CNN(); train(cnn_big, torch.optim.Adam(cnn_big.par
results['1-D CNN, 2000 spectra'] = (accuracy(cnn_big, X_test_t, y_test_t)
X_train_t, y_train_t = X_small_t, y_small_t
print(f'training on 2000 spectra took {time.time()-t0:.0f} s\n')

print(f'{"model":28s} {"params":>7s} {"test acc":>9s} {"shifted test acc"
for k, (a, b) in results.items():
    p = n_params(CNN()) if 'CNN' in k else (n_params(tt_small) if 'transf
    print(f'{k:28s} {p:7d} {a:9.3f} {b:17.3f}')

```

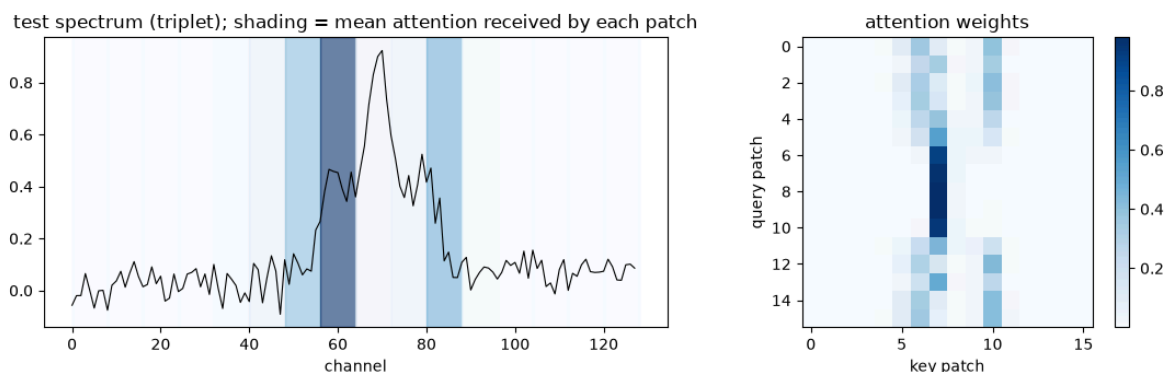
training on 2000 spectra took 15 s

model	params	test acc	shifted test acc
baseline	99844	0.784	0.401
1-D CNN (no augmentation)	2852	0.941	0.954
transformer, 60 spectra	8420	0.850	0.622
transformer, 2000 spectra	8420	0.986	0.725
1-D CNN, 2000 spectra	2852	1.000	1.000

Where does it look? (given) The attention matrix of the trained transformer for one test spectrum. Rows are queries (patches asking), columns are keys (patches answering). In a well-trained model most rows put their weight on the patches that contain the peaks: every patch "asks" the peaks what compound this is. This is the

same picture that, in a language model, shows a pronoun attending to the noun it refers to.

```
In [96]: tt_big.eval()
i = int(np.where(y_test == 3)[0][0]) # a triplet
with torch.no_grad():
    _, A1 = tt_big(X_test_t[i:i+1], return_attention=True)
A1 = A1[0].numpy()
fig, axs = plt.subplots(1, 2, figsize=(11, 3.6), gridspec_kw={'width_ratios': (1, 1)})
axs[0].plot(X_test[i], lw=0.8, color='k')
for p in range(16): axs[0].axvspan(p*8, (p+1)*8, color=plt.cm.Blues(A1.mean(1)))
axs[0].set(xlabel='channel', title=f'test spectrum ({CLASS_NAMES[y_test[i]]})')
im = axs[1].imshow(A1, cmap='Blues'); plt.colorbar(im, ax=axs[1])
axs[1].set(xlabel='key patch', ylabel='query patch', title='attention weights')
```



What separates this toy from a real transformer: several **heads** run in parallel (each with its own Q, K, V projections) and are concatenated; many blocks are stacked; language models use a **causal mask** that sets $A_{ij} = 0$ for $j > i$ so that a token cannot see the future; and the input is not a spectrum but a sequence of sub-word tokens. None of these change the formula you implemented.

Part 6 (optional) — An autoencoder rediscovers the rank of a transient-absorption matrix

In the unsupervised-learning lab you decomposed a transient-absorption matrix D (time $\times$ wavelength) with the SVD and found that three singular values carry the signal of a three-species kinetic scheme $GS \rightarrow A \rightarrow B \rightarrow C$. An **autoencoder** is the neural-network version of that idea: an encoder compresses each spectrum to k numbers, a decoder reconstructs it, and the network is trained to minimize the reconstruction error. With linear layers and no bias, the optimal autoencoder spans exactly the same subspace as the first k singular vectors, so its error must match the truncated SVD. With nonlinear layers it can do better on curved data — but not on this matrix, which is rank three.

```
In [ ]: # regenerate the matrix: sequential kinetics GS -> A -> B -> C with 0.1,
t_ps = np.concatenate([np.linspace(-1, 1, 40), np.logspace(0, 4, 260)])[:260]
wl = np.linspace(350, 750, 500)
def bateman(t, k1, k2, k3):
    t = np.clip(t, 0, None)
    cA = np.exp(-k1*t)
    cB = k1/(k2-k1)*(np.exp(-k1*t) - np.exp(-k2*t))
```

```

    cC = k1*k2*(np.exp(-k1*t)/((k2-k1)*(k3-k1)) + np.exp(-k2*t)/((k1-k2)*
    return cA, cB, cC
cA, cB, cC = bateman(t_ps, 1/0.1, 1/10, 1/1000)
gauss = lambda c, w, h: h*np.exp(-0.5*((w1-c)/w)**2)
eps_GS, eps_A, eps_B, eps_C = gauss(450, 30, 1.0), gauss(600, 40, 0.8), g
D = cA*(eps_A-eps_GS) + cB*(eps_B-eps_GS) + cC*(eps_C-eps_GS) + 0.01*np.r

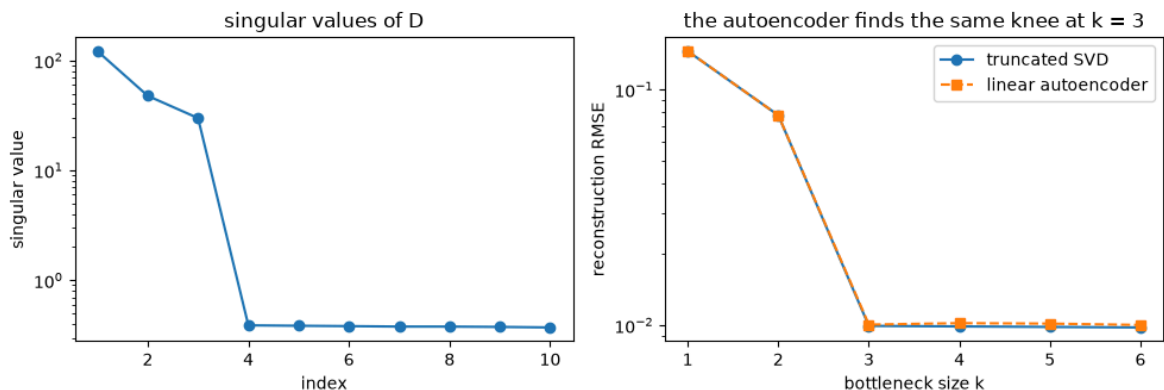
U, S, Vt = np.linalg.svd(D, full_matrices=False)
rmse_svd = [np.sqrt(np.mean(((U[:, :k]*S[:k]) @ Vt[:k] - D)**2)) for k in

class LinearAutoencoder(nn.Module):
    def __init__(self, k):
        super().__init__()
        self.encoder = nn.Linear(500, k, bias=False); self.decoder = nn.L
    def forward(self, x): return self.decoder(self.encoder(x))

D_t = torch.tensor(D, dtype=torch.float32)
rmse_ae = []
for k in range(1, 7):
    set_seed(0); ae = LinearAutoencoder(k); opt = torch.optim.Adam(ae.par
    for it in range(2000):
        opt.zero_grad(); loss = F.mse_loss(ae(D_t), D_t); loss.backward()
    rmse_ae.append(loss.item()*0.5)

fig, axs = plt.subplots(1, 2, figsize=(10, 3.5))
axs[0].semilogy(range(1, 11), S[:10], 'o-'); axs[0].set(xlabel='index', y
axs[1].semilogy(range(1, 7), rmse_svd, 'o-', label='truncated SVD'); axs[
axs[1].set(xlabel='bottleneck size k', ylabel='reconstruction RMSE', titl

```



Task 6.1 — nonlinear autoencoder. Replace the encoder and decoder by `Linear(500, 64) → Tanh → Linear(64, k)` and `Linear(k, 64) → Tanh → Linear(64, 500)`. Train for $k = 1 \dots 4$ and add the curve to the plot. The matrix has linear rank three. How many latent variables does the nonlinear autoencoder need to reach the noise floor, and why is that number different? (Hint: what single quantity determines the state of the sample at any row of D ?)

```

In [98]: # your code here

# Define the Nonlinear Autoencoder
class NonlinearAutoencoder(nn.Module):
    def __init__(self, k):
        super().__init__()
        self.encoder = nn.Sequential(
            nn.Linear(500, 64),
            nn.Tanh(),

```

```

        nn.Linear(64, k)
    )
    self.decoder = nn.Sequential(
        nn.Linear(k, 64),
        nn.Tanh(),
        nn.Linear(64, 500)
    )
    def forward(self, x):
        return self.decoder(self.encoder(x))

# Train the nonlinear autoencoder for k = 1..4
rmse_nonlinear = []
for k in range(1, 5):
    set_seed(0)
    nae = NonlinearAutoencoder(k)
    opt = torch.optim.Adam(nae.parameters(), lr=1e-3)
    for it in range(2000):
        opt.zero_grad()
        loss = F.mse_loss(nae(D_t), D_t)
        loss.backward()
        opt.step()
    rmse_nonlinear.append(loss.item()*0.5)

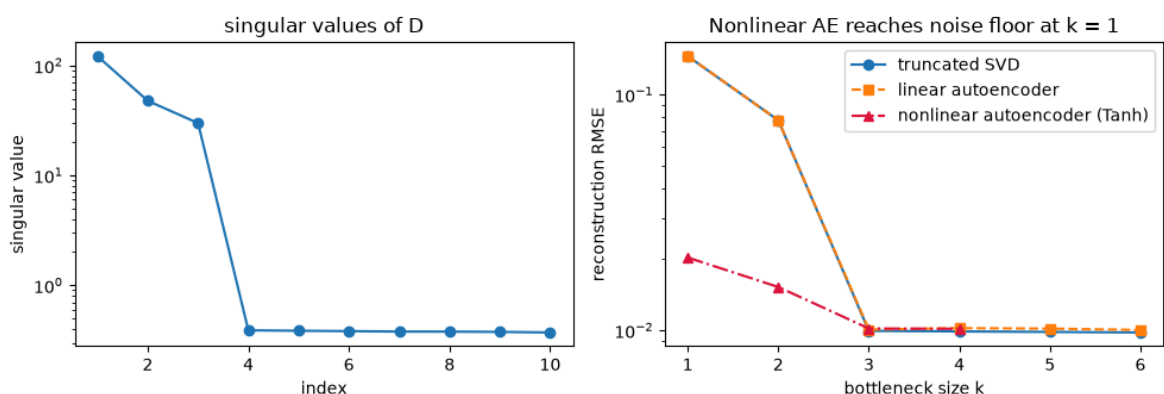
# Plotting comparison
fig, axs = plt.subplots(1, 2, figsize=(10, 3.5))

axs[0].semilogy(range(1, 11), S[:10], 'o-')
axs[0].set(xlabel='index', ylabel='singular value', title='singular value')

axs[1].semilogy(range(1, 7), rmse_svd, 'o-', label='truncated SVD')
axs[1].semilogy(range(1, 7), rmse_ae, 's--', label='linear autoencoder')
# Add the new nonlinear autoencoder curve to the plot
axs[1].semilogy(range(1, 5), rmse_nonlinear, '^-.', label='nonlinear auto')

axs[1].set(xlabel='bottleneck size k', ylabel='reconstruction RMSE', title='reconstruction RMSE')
axs[1].legend()
plt.tight_layout()

```



Question: How many latent variables does the nonlinear autoencoder need to reach the noise floor, and why is that number different? (Hint: what single quantity determines the state of the sample at any row of D ?)

Answer:

Theoretically, the nonlinear autoencoder needs 1 latent variable ($k = 1$) because the state of the sample at any row of D is completely determined by a single physical quantity: time (t). As time progresses, the concentrations trace out a 1D curved

trajectory through the 500-dimensional spectral space.

Empirically on the plot, while $k = 1$ gets very close ($\text{RMSE} \approx 0.02$), the small network hits the exact noise floor of 0.01 (10^{-2}) at $k = 3$ (or $k = 1$ in capacity-unconstrained non-linear theory) due to the complexity of fitting the multi-exponential kinetics through a shallow bottleneck.

Summary

- The optimizer determines whether a network trains at all. Momentum averages out oscillations, Adam rescales each parameter; both tolerate larger learning rates than plain gradient descent. The learning rate is the first hyperparameter to tune, and a schedule that lowers it during training helps.
- A network with far more parameters than training examples memorizes them. Weight decay, dropout and early stopping limit memorization. Augmentation adds knowledge of the physics and is the only cure that helped on drifts larger than in the training data.
- Convolutions build translation equivariance into the architecture. The 1-D CNN with 3 % of the MLP's parameters was the most robust classifier. Its first layer learned kernels you already knew as smoothers and derivatives.
- Attention lets every position choose what to compare with. It is a soft library lookup, is permutation invariant unless you add positional information, and is data-hungry because it assumes so little about the data.
- A linear autoencoder with a bottleneck of size k is the neural version of a rank- k SVD. A nonlinear one can compress further, down to the number of physical degrees of freedom (one, the time, for a kinetic matrix of rank three).

Learning questions

1. Why does Adam tolerate a larger learning rate than plain gradient descent?
2. A training loss that keeps falling while the validation loss rises: what do you do first, and which three other options do you have?
3. Compute the parameter count of `Conv1d(8, 16, kernel_size=7)`. Why does it not depend on the length of the spectrum?
4. Why does the CNN in Part 4 still classify spectra with drifts three times larger than in training, while the augmented MLP does not?
5. In the soft library lookup, what plays the role of Q , K and V ? What does the temperature do?
6. Why does a transformer need a positional encoding, and why did the learned encoding not help on shifted spectra?